CORE JAVA SERIES — DAY 11

# Immutable Classes, Deep Copy & the Object Class

What makes an object immutable, why a reference field breaks it, shallow vs deep copy with defensive copying — then the Object class at the root of every Java class: toString, equals, hashCode, getClass, clone and the rest.

## Immutable Objects

An object is **immutable** if nothing about it can change after it is created — neither its variables (instance variables) nor its behaviour (methods). If either can change, it is **mutable**.

An ordinary class is mutable in **two** ways. Take a `Student` with `age`, `name`, a constructor, getters and setters:

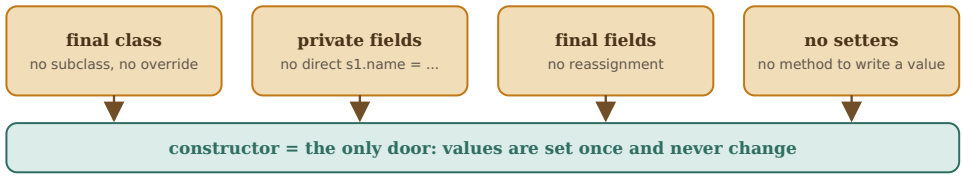
- **Values change** — `s1.setName("Rohit")` overwrites the name; the age can be changed the same way.
- **Behaviour changes** — a subclass can `extends Student` and override a method such as `markAttendance()`, changing how the object behaves.

```
class Student {
    int age; String name;
    Student(int age, String name) { this.age = age; this.name = name; }
    void setName(String name) { this.name = name; }     // values can change
    void markAttendance() { /* ... */ }
}
class CSStudent extends Student {
    @Override void markAttendance() { /* ... */ }       // behaviour can change
}
Student s1 = new Student(28, "Aditya");
s1.setName("Rohit");                                    // s1 has been mutated
```

### Rules for an immutable class (first attempt)

Each rule shuts one door through which the object could be changed:

| Rule | What it blocks |
|---|---|
| **Make the class `final`** | No one can extend it and override its methods, so behaviour is frozen. |
| **Make every instance variable `private`** | No direct writes like `s1.name = ...` from outside. |
| **Make them `final` too** | Once assigned, a field can never be reassigned. |
| **Remove all setters** | No method exists that can write a value. Getters are fine — they only read. |
| **Initialise only in the constructor** | A `final` field gets its single assignment there, at creation time. |




Four restrictions close every path; the constructor is the single entry point for values.

```
final class Student {                          // rule 1
    private final int age;                         // rules 2 and 3
    private final String name;

    Student(int age, String name) {                // rule 5: the only assignment
        this.age = age;
        this.name = name;
    }
    public int getAge()     { return age; }        // getters only
    public String getName() { return name; }
}
```

Now nothing can be changed: `s1.name` is not visible, no `setName` exists, and writing `class CSStudent extends Student` fails with *"The type CSStudent cannot subclass the final class Student"*, so its methods cannot be overridden either. The fields are an `int` and a `String` — a primitive is just a value, and `String` is immutable by itself (the reason comes when Strings are studied) — so this class is truly immutable.

## When a Field Is Another Object

A college is not a single string — it has a name, an address and more — so it gets its own class. `College` is an ordinary class here: fields without access modifiers, no setters needed because anything in the package can write them. `Student` now holds a `College` and still follows all five rules:

```
class College {                                  // plain, mutable class
    String name; String address;
    College(String name, String address) { this.name = name; this.address = address; }
}

final class Student {
    private final int age;
    private final String name;
    private final College college;

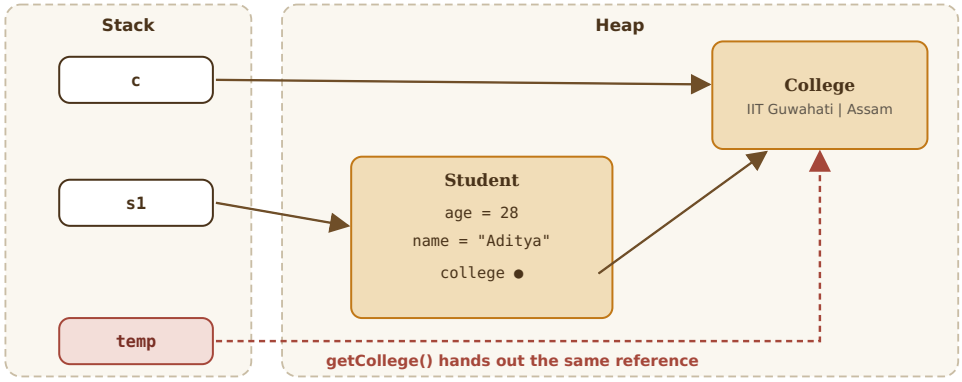
    Student(int age, String name, College college) {
        this.age = age; this.name = name;
        this.college = college;                  // stores the reference it was given
    }
    public College getCollege() { return college; }   // hands the reference out
}

College c = new College("IIT Guwahati", "Assam");
Student s1 = new Student(28, "Aditya", c);
System.out.println(s1.getCollege().name);        // IIT Guwahati
s1.getCollege().name = "IIT Bombay";             // allowed! no setter was needed
System.out.println(s1.getCollege().name);        // IIT Bombay  -> the Student changed
```

**Every rule was followed, yet the Student changed.** The class is **not purely immutable** — the earlier version was safe only because it had no reference to a mutable object.

### Why it happens

`new College(...)` creates one object in the heap, and `c` points to it. The `Student` object stores that same address in its `college` field. When `getCollege()` returns, the caller receives (in a temporary variable) the **same address** — the actual College inside the Student — and can write to it through the dot operator.




c, the Student's field and the value returned by getCollege() all point to a single College object.

Copying only the *reference* instead of the object is a **shallow copy**: two names, one object. It is the same idea as the reference assignment `s1 = s2` studied earlier.

## Active Recall — Part 1

**Q1. When is an object called immutable?**

A. When neither its variables nor its behaviour can change after creation.

**Q2. List the rules for an immutable class.**

A. final class; private final fields; no setters; values assigned only in the constructor (getters are fine).

**Q3. Why does making the class final matter?**

A. Otherwise a subclass could override methods and change the behaviour.

**Q4. Why are int and String fields safe, but a College field is not?**

A. A primitive is only a value and String is immutable; College is a mutable object whose reference can be handed out.

**Q5. Why can a getter break immutability here?**

A. It returns the reference to the real College, so the caller can modify it with no setter.

## Making It Truly Immutable

### Fix 1 — make the referenced class immutable too

Apply the same rules to `College`: `final` class, `private final` fields, only getters. Then `getCollege()` can safely return the real object because nobody can write into it. In general, **every non-primitive field must itself be immutable**; primitives and Strings are already safe.

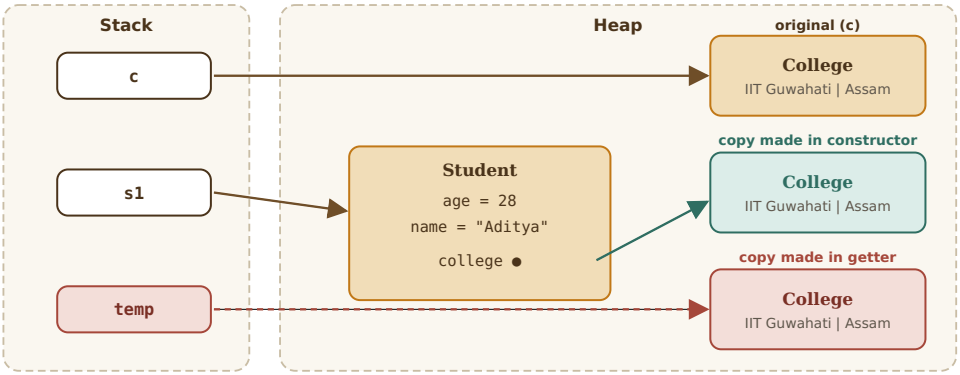
### Fix 2 — defensive copies (when you can't change College)

Sometimes `College` comes from a library and cannot be edited. Then `Student` must never share its real College with anyone, which needs a **defensive copy** in two places: when the object is built and when it hands one out.

```
final class Student {
    private final int age;
    private final String name;
    private final College college;

    Student(int age, String name, College college) {
        this.age = age; this.name = name;
        this.college = new College(college.name, college.address);   // copy IN
    }
    public College getCollege() {
        return new College(college.name, college.address);           // copy OUT
    }
}

College c = new College("IIT Guwahati", "Assam");
Student s1 = new Student(28, "Aditya", c);
s1.getCollege().name = "IIT Bombay";            // changes only a throw-away copy
System.out.println(s1.getCollege().name);       // IIT Guwahati
System.out.println(c.name);                     // IIT Guwahati
```




The Student keeps a private College that is never exposed; callers only ever receive fresh copies.

- **Copy in (constructor):** without it, the outside code still holding `c` could edit `c.name` later and the change would leak straight into the Student.
- **Copy out (getter):** whoever calls `getCollege()` gets a new object, so editing it never touches the one stored inside.

> **Shallow copy vs deep copy.** Shallow: only the reference is copied, so both sides share one object. **Deep copy:** a separate new object is created for every non-primitive part, and you hand out the reference to the copy — never the same object. Here we never shared the stored College at all, which is exactly a deep copy.

### Why immutable objects matter

When several threads run the same code at once, a shared object that can change leads to **race conditions**. An immutable object guarantees that nobody can change its variables or methods, so that whole class of problem disappears. This becomes clearer once multithreading is covered.

## Active Recall — Part 2

**Q1. What are the two ways to fix the College problem?**
A. Make College immutable too, or make defensive copies in the constructor and the getter.

**Q2. What is a defensive copy?**
A. Creating a new object from an incoming or outgoing reference so the original is never shared.

**Q3. Why copy in the constructor as well as in the getter?**
A. Otherwise the caller who passed in the original can still change it later.

**Q4. Define shallow copy and deep copy.**
A. Shallow copies the reference (shared object); deep creates separate new objects for non-primitive parts.

**Q5. Why do immutable objects help with threads?**
A. Nothing can change them, so race conditions on their data cannot occur.

## The Object Class — Root of Every Class

Java contains a class literally named `Object`. It lives in the `java.lang` package, which is imported automatically (the same reason `System` needs no import), so it is always available.

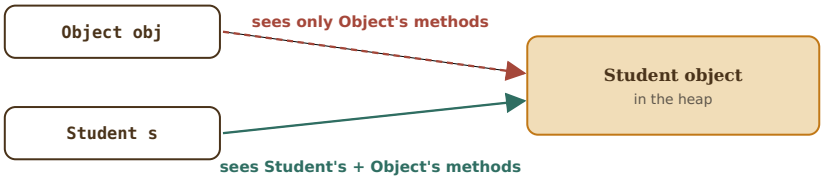
**Every class in Java inherits from Object**, directly or indirectly. Write `class Student` and the compiler quietly adds `extends Object`. If `Student extends Human` and `Human extends Animal`, then `Animal` (which has no parent of its own) extends `Object`, so the chain still ends there. Java has single inheritance, so there is exactly one chain per class, and it always finishes at Object — for your own classes and for built-in ones like `Integer`.




Whatever the length of the hierarchy, the top is always Object — String, Integer, arrays and every other class end there too.

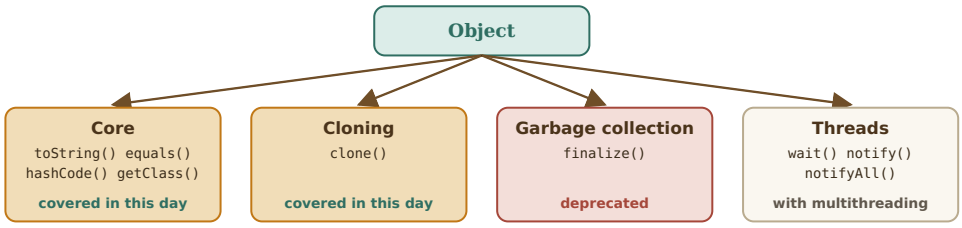
### Why Java needs a root class

- **Common behaviour.** Behaviours every class needs can be written once in Object, and inheritance hands them to every class in Java.
- **Polymorphism.** A parent reference can point to a child object, so an `Object` reference can point to *any* object: `Object obj = new Student();`. Through `obj` only Object's own methods are callable, but it means there is always a type that accepts anything.




Same object, two references: the reference type decides which methods are visible.

### What Object contains




Methods of Object grouped by purpose. This day covers the core four and clone.

## toString()

`toString()` returns a **string representation** of an object. Its signature is `public String toString()`. Because every class inherits it, it can be called on any object, even without writing anything — the default prints the class name, an `@` sign and a hexadecimal number derived from the object's hash code, such as `Student@1b6d3586`. That number changes between machines and runs but identifies the object.

To control what appears, override it. It must **return** a `String` — printing inside it is a compile error because the return type is `String`.

```
class Student {                     // "extends Object" is added automatically
    String name; int age;

    @Override
    public String toString() {
        return name + ", " + age;
    }
}

Student s1 = new Student();
s1.name = "Aditya"; s1.age = 28;
System.out.println(s1.toString());   // Aditya, 28
System.out.println(s1);              // Aditya, 28  (println calls toString() itself)
```

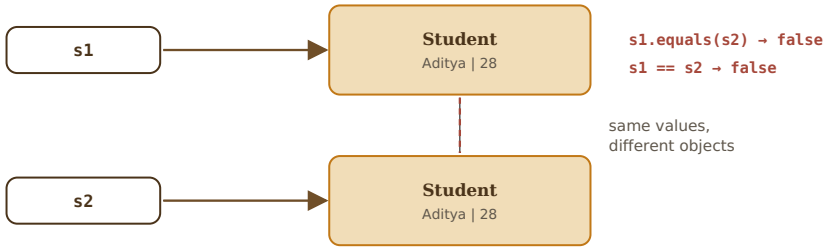
`System.out.println(obj)` converts its argument to a string through `toString()`, so writing `.toString()` yourself is optional there. Writing `extends Object` explicitly is optional too — the output is identical.

## equals()

`equals()` compares two objects and returns a `boolean`: `public boolean equals(Object obj)`. The parameter is an `Object` because Object has no idea which classes will extend it later, yet it must accept any of them.

The **default** implementation compares **references**: inside Object it is simply `return (this == obj);`. Two separately created objects are therefore never equal, however identical their contents — `equals()` behaves exactly like `==`.

```
Student s1 = new Student(); s1.name = "Aditya"; s1.age = 28;
Student s2 = new Student(); s2.name = "Aditya"; s2.age = 28;
System.out.println(s1.equals(s2));   // false  (two objects, same values)
System.out.println(s1 == s2);        // false  (default equals acts like ==)
```



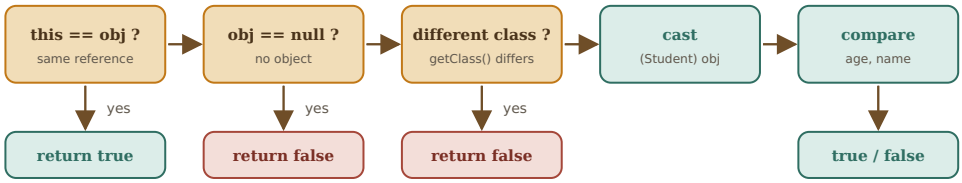

Identical contents do not matter to the default equals(); only whether both references point to one object.

### Overriding equals() properly

To compare by **value**, override it. The parameter must stay `Object`: if you write `equals(Student s)`, the `@Override` annotation reports *"must override or implement a supertype method"*, because that is a different method, not an override. So the body casts the argument itself — after making sure the cast is safe:

```
import java.util.Objects;

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;                          // 1. same object
    if (obj == null) return false;                         // 2. nothing to compare
    if (getClass() != obj.getClass()) return false;        // 3. different class
    Student s = (Student) obj;                             // 4. cast is now safe
    return age == s.age && Objects.equals(name, s.name);   // 5. compare the values
}
```




Cheap exits first; only an object of the same class with matching fields reaches the final comparison.

| Check | What it protects against |
|---|---|
| `this == obj` → **true** | Same reference means the same object, so the remaining work can be skipped. |
| `obj == null` → **false** | Reading `s.name` / `s.age` on null throws `NullPointerException`. |
| **different** `getClass()` → **false** | Passing, say, an `Integer` and casting it to `Student` throws `ClassCastException`. |

With this override, `s1.equals(s2)` prints `true`, while `s1 == s2` still prints `false`, because `==` always compares references.

> `name` is a String, i.e. a reference, so compare it with `equals` (here the null-safe `Objects.equals`), not `==`. A `==` only seems to work in tiny examples because identical string literals share one object.

## Active Recall — Part 3

**Q1. Where does Object live, and why need it not be imported?**
A. In java.lang, which is imported automatically.

**Q2. Why can an Object reference point to a Student?**
A. Student inherits Object, and a parent reference can refer to a child object (polymorphism).

**Q3. What does the default toString() print?**
A. ClassName@hexNumber, e.g. Student@1b6d3586.

**Q4. What does the default equals() compare?**
A. References (this == obj), not values.

**Q5. Why must the equals() parameter be Object, and what two checks come before casting?**
A. Otherwise it is not an override. Check for null (NullPointerException) and for the same class (ClassCastException).

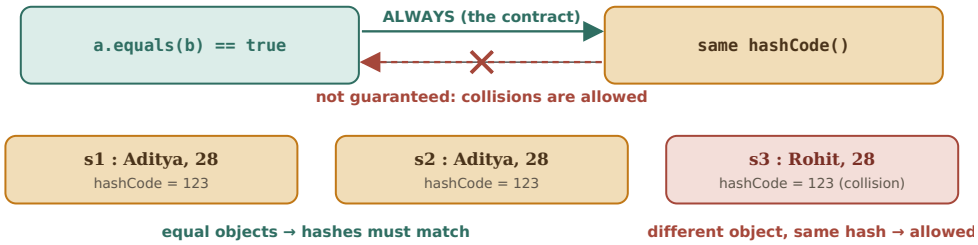
## hashCode()

`hashCode()` turns an object into an **integer**: `public int hashCode()`. A hash function always gives the same number for the same input — "Aditya" might always map to one number, "Rohit" to another — and it works for any object, not just Strings. The hash-based collections (`HashMap`, `HashSet`, `Hashtable`, studied later) depend on it to place and find objects. The default value is derived from the object's identity; the hexadecimal text in the default `toString()`

is this number.

### The equals–hashCode contract

- If two objects are equal (`a.equals(b)` is `true`), their hash codes **must** be equal. This rule must never be broken.
- The reverse **need not** hold: two different objects may share a hash code (a **collision**), because the formula can produce the same number for different data.




Equal objects must share a hash; sharing a hash does not make objects equal.

Why it matters: a hash-based collection uses the hash to decide where an object goes and where to look for it. If equal objects had different hashes they would land in different places and the collection could not recognise them as the same — the details come with the collections framework.

With the **default** `equals()` the contract holds automatically: two objects are "equal" only when they are the very same object, which naturally has one hash. The moment you **override** `equals()`**, you must override** `hashCode()` **as well**. Without it:

```
System.out.println(s1.equals(s2));                  // true  (equals is overridden)
System.out.println(s1.hashCode() == s2.hashCode());   // false <- contract broken!
```

Fix it by building the hash from the **same fields** equals() uses. A classic hand-written version starts from a non-zero prime and folds each field in, multiplying by another prime — primes spread the results out, so different objects collide less often:

```
@Override
public int hashCode() {
    int result = 17;                                        // non-zero prime
    result = result * 31 + age;                             // fold in age
    result = result * 31 + (name == null ? 0 : name.hashCode());   // fold in name
    return result;
}
```

- `String` already overrides `hashCode()`, so `name.hashCode()` is reused rather than recomputed.
- The `name == null ? 0 : ...` guard avoids a `NullPointerException` when the name is null.

Java ships the same logic as a one-liner:

```
import java.util.Objects;

@Override
public int hashCode() { return Objects.hash(name, age); }   // same result, far shorter
```

> `Object` (java.lang) is the root class; `Objects` (java.util) is a separate **utility** class with helpers like `hash()` and `equals()`. One letter apart, two different classes — the same pattern as `Array` vs `Arrays`.

## getClass() and instanceof

`getClass()` returns the **runtime class** of an object. Its signature is `public final Class<?> getClass()` — `final`, so it cannot be overridden (nobody may fake an object's class). The return value is an object of Java's own class named `Class` (it is the subject of reflection, covered later); calling `.getName()` on it gives the class name as a string.

```
Student s1 = new Student();
System.out.println(s1.getClass().getName());   // Student  (with package prefix if it has one)
```

The `instanceof` operator is related but answers a different question: is the object an instance of a class *or any of its subclasses*?

```
class Animal { }
class Dog extends Animal { }

Animal a = new Animal();
Animal d = new Dog();                       // parent reference, child object

a.getClass().getName();    // Animal
d.getClass().getName();    // Dog          (the actual object, not the reference type)
a instanceof Animal;       // true
d instanceof Animal;       // true         (Dog is a subclass of Animal)
a instanceof Dog;          // false        (an Animal is not a Dog)
```




Both references are of type Animal; the objects behind them are what getClass() reports.

| | getClass() | instanceof |
|---|---|---|
| **Answers** | The exact class of the object created in memory. | Whether the object is that class or any subclass of it. |
| **Example use** | `equals()`, to demand the very same class. | Checking that an object can be treated as a given type. |

Since Object is the root, `s1 instanceof Object` is `true` for any object.

## Active Recall — Part 4

**Q1. State the equals-hashCode contract.**
A. Equal objects must have equal hash codes; the reverse is not required.

**Q2. Why does overriding equals() force you to override hashCode()?**
A. Otherwise two equal objects can get different hashes, breaking hash-based collections.

**Q3. Why use primes like 17 and 31 in a hash formula?**
A. They spread results out so different objects collide less often.

**Q4. Why can getClass() not be overridden?**
A. It is final — it reports the true class of the object.

**Q5. For Animal d = new Dog(): what do d.getClass().getName() and d instanceof Dog give?**
A. "Dog" and true.

## clone()

`clone()` creates a copy of an object. Its signature is `protected Object clone() throws CloneNotSupportedException`. Being `protected`, it cannot simply be called from anywhere; a class opts in by overriding it and delegating to Object's version with `super.clone()`. It returns `Object`, so the result must be cast back to the real type.

```
class Student implements Cloneable {               // opt-in marker (see below)
    String name; int age;

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();                          // Object's built-in copying
    }
}

public static void main(String[] args) throws CloneNotSupportedException {
    Student s1 = new Student(); s1.name = "Aditya"; s1.age = 28;
    Student s3 = (Student) s1.clone();                 // a new object in the heap
    System.out.println(s3.name + ", " + s3.age);       // Aditya, 28
}
```

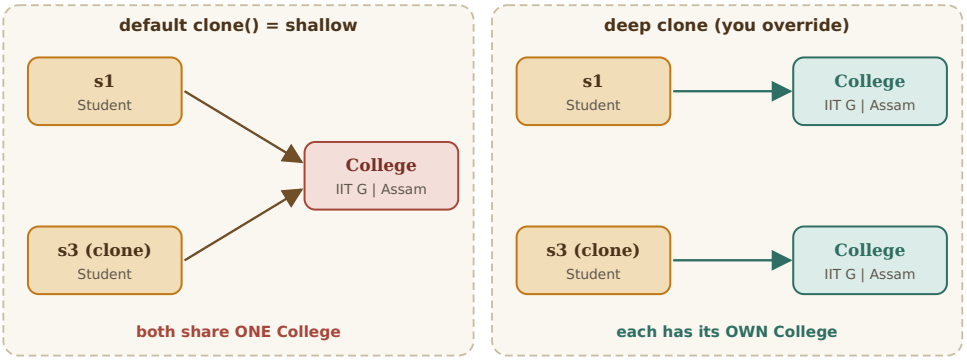
The `throws` clauses are required because `clone()` declares a checked exception; exception handling is covered later, so just include them for now. Access is shown within one package — to call `clone()` from a different package, make the override `public`.

### Why Cloneable?

Not every object should be copyable — think of a database connection or a thread; a duplicate would give undesired results. So Java makes cloning opt-in: the class must `implements Cloneable`. Without it, `super.clone()` throws `CloneNotSupportedException` at run time. `Cloneable` has **no methods at all** — an empty interface used purely as a tag. Such empty tag interfaces are called **marker interfaces**, and they exist only to unlock a capability when a class declares them. Conceptually Object's clone does this:

```
// the idea behind Object.clone()
if (this instanceof Cloneable) {
    // make a copy
} else {
    throw new CloneNotSupportedException();
}
```

**The built-in clone() makes a shallow copy.** Primitive fields are copied by value, but a reference field (like the College above) is copied as a reference, so the original and the clone share it. For a **deep copy**, override `clone()` and create fresh copies of the non-primitive fields yourself.



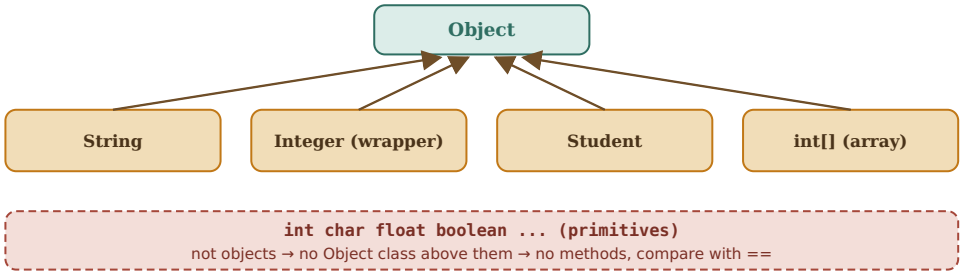

Default clone() shares the College; an overridden deep clone gives each Student its own.

## finalize(), wait(), notify() and Friends

- `finalize()` was run by the garbage collector just before reclaiming an object's memory. It is now **deprecated**: whether and when it runs is up to the collector, so it is unpredictable, unsafe and unreliable. It stays in Object only for legacy reasons; do not use it.
- `wait()`, `notify()`, `notifyAll()` coordinate threads. They only make sense with multithreading and are covered there.

## Arrays, Primitives and the Object Class

- **Arrays are objects.** `int[] arr = new int[5];` creates the array in the heap with `arr` as a reference, so Object's methods work on arrays too.
- **Primitives are not objects.** `int`, `char`, `float` and the like have no Object above them and no methods; compare them with `a == b`, never `a.equals(b)`.
- **Wrapper classes are objects.** `Integer`, `Character`, `Float` and friends are real classes, so they extend Object like any other.
- **Similar names, different things:** `Object` vs `Objects`, and the `int[]` array type vs the `Arrays` utility class.




Everything that lives in the heap descends from Object; primitives stay outside the hierarchy.

## Active Recall — Part 5

**Q1. Why is clone() protected, and what must a class do to allow cloning?**
A. Cloning is opt-in: override clone() calling super.clone() and implement Cloneable.

**Q2. What happens if a class does not implement Cloneable?**
A. super.clone() throws CloneNotSupportedException at run time.

**Q3. What is a marker interface?**
A. An empty interface that only tags a class to unlock a capability, e.g. Cloneable.

**Q4. Is the default clone() shallow or deep? How do you get a deep one?**
A. Shallow. Override clone() and copy the non-primitive fields yourself.

**Q5. Why is finalize() not used, and do int values have Object methods?**
A. It is deprecated — unpredictable, unsafe, unreliable. Primitives are not objects, so no.